ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Report

on

Voting Eligibility Checker

by

Lokesh Rathod

150096725094

1. Introduction

Case Study No.132 which is to build a complete Voting Eligibility System in C++. The program addresses a realistic civic scenario: given a person's full name, date of birth, and citizenship status, the system must determine whether they are eligible to vote in India. The implementation demands multiple non-trivial engineering decisions — calculating exact age from a calendar date, generating a unique Voter ID, persisting records to a file, and defending the program against erroneous user input.

Scope of this Report: This document provides a structured analysis of the case study, covering the problem context, design rationale, a function by function code walkthrough, testing scenarios, complexity assessment, strengths, limitations, and learning outcomes — to demonstrate a thorough understanding of both the programming constructs used and the real world considerations behind them.

2. Problem Statement

The Representation of the People Act, 1951 stipulates that a person is eligible to vote in India if they are at least 18 years of age and are an Indian citizen. Translating this legal rule into software requires more than a simple comparison, it requires validated data collection, precise date arithmetic and reliable output generation.

Core Requirements:

- Accept the user's full name, including spaces, using getline().
- Accept Date of Birth in DD MM YYYY format and compute the exact current age.
- Accept citizenship input in a case insensitive manner.
- Display a clear eligibility verdict — ELIGIBLE or NOT ELIGIBLE.
- For eligible voters: generate a unique Voter ID (IND-VTR-XXXX) and store the record to voter_records.txt.
- Allow the user to check multiple people via a menu-driven do-while loop.
- Handle all invalid inputs gracefully without crashing.

3. Program Design & Architecture

The program follows a procedural architecture organised into five distinct helper functions plus main(), each with a single responsibility. This separation of concerns keeps the code readable, testable, and maintainable.

Function	Responsibility
toLowerCase(string)	Converts a string to lowercase for case-insensitive comparison.
generateVoterID()	Seeds rand() once and returns a unique "IND-VTR-XXXX" string.
calculateAge(d, m, y)	Computes exact age using struct tm, adjusting for birthday passage.
getValidatedDOB()	Loops until a valid DD MM YYYY date is entered; returns computed age.
storeVoter(name, age, id)	Appends voter record to voter_records.txt via ofstream + ios::app.
returnToMainMenu()	Waits for 0 input before returning to the do-while menu loop.
main()	Orchestrates the do-while menu, input collection, and eligibility logic.

Control Flow Overview:

The program's top-level control flow is a do-while loop in main(), which displays a two-option menu and repeats until the user selects Exit (option 2). Inside the loop, choosing option 1 triggers the full eligibility check sequence:

1. Display menu and read choice with input validation.
2. Prompt for full name using getline() (preceded by cin.ignore() to flush the leftover newline).
3. Call getValidatedDOB() which loops until a valid DD MM YYYY is entered and returns the computed age.
4. Prompt for citizenship, pass through toLowerCase().
5. Evaluate: if age >= 18 AND citizenship == "indian" then ELIGIBLE branch; else NOT ELIGIBLE branch.
6. ELIGIBLE path: generate Voter ID, display result, call storeVoter().
7. NOT ELIGIBLE path: display specific reason — age or citizenship.
8. Call returnToMainMenu(): user presses 0 to loop back to the menu.

4. Code Analysis — Function by Function

4.1 toLowerCase(string str): This utility function accepts a string by value and uses the STL transform() algorithm with the ::tolower function pointer to convert every character to its lowercase equivalent before returning the modified copy.

4.2 generateVoterID(): This function uses a static boolean flag to ensure srand(time(0)) is called exactly once per program run. rand() % 9000 + 1000 produces an integer in [1000, 9999], concatenated with the prefix "IND-VTR-" using to_string().

4.3 calculateAge(int day, int month, int year): This is the most algorithmically significant function. It retrieves the current date using time(0) and localtime(), subtracts the birth year from the current year, then decrements by 1 if the birthday in the current year has not yet passed.

4.4 getValidatedDOB(): This function implements a validation loop that repeatedly prompts for DD MM YYYY until the input passes three checks:

(1) numeric parse succeeds (no cin.fail()),

(2) day/month/year are in plausible ranges,

(3) the computed age is between 0 and 120.

4.5 storeVoter(string name, int age, string voterID): This function opens voter_records.txt in append mode (ios::app) so each program run adds new records without overwriting existing data, writes a formatted record, and closes the file.

4.6 returnToMainMenu(): A guarded pause loop that requires the user to input 0 before returning. This prevents the menu from immediately refreshing and hiding the eligibility result in a terminal UI.

4.7 main() — Orchestration & the cin.ignore() Idiom

cin.ignore() before getline(): This is a critical idiom. After reading the integer choice with cin >>, a newline character '\n' remains in the input buffer. Without consuming it, the subsequent getline() call immediately reads an empty string. Placing cin.ignore() between the two reads correctly discards the newline and allows getline() to wait for the user's next input.

do-while vs while: The do-while loop guarantees the menu is displayed at least once — semantically correct for an interactive menu application. A plain while loop would require either initialising choice to a sentinel value or duplicating the display code before the loop.

5. Strengths & Limitations

Strengths:

- DOB-based age calculation is more accurate and tamper-proof than accepting a raw age input.
- Case-insensitive citizenship check is user-friendly and matches real-world input behaviour.
- Comprehensive input validation using the standard cin.fail() + cin.clear() + cin.ignore() recovery pattern.
- Reason specific failure messages (age vs. citizenship) improve user communication quality.
- do-while loop guarantees at least one menu display — semantically correct for interactive CLIs.

Limitations & Improvement Opportunities

Limitation	Suggested Improvement
No leap-year / impossible date check (31 Apr accepted)	Implement a days-in-month lookup with leap-year handling
rand() can repeat IDs if called in the same second	Use <random> std::mt19937 or a DB sequence for unique IDs
No error check in storeVoter() on file.is_open()	Add: if (!file) { cerr << "Cannot open file"; return; }
Records are plain text — no dedup or search	Use CSV with unique constraints or SQLite for data integrity
Only 9,000 possible Voter IDs (4-digit format)	Extend to alphanumeric format or UUID for production scale

6. C++ Concepts Demonstrated

Concept	Function / Location	Specific Feature
String manipulation	toLowerCase()	std::transform(), ::tolower, pass-by-value
Pseudo-random numbers	generateVoterID()	rand(), srand(time(0)), static flag
Date & time arithmetic	calculateAge()	time_t, struct tm, localtime(), field offsets
File I/O (write)	storeVoter()	ofstream, ios::app, file.close()
Stream error recovery	getValidatedDOB(), main()	cin.fail(), cin.clear(), cin.ignore()
Control flow — loops	All validation functions	while(true) with break, do-while
Modular functions	All 6 helpers	Single responsibility, return by value
Conditional logic	main() eligibility check	if-else, logical && operator
Static local variables	generateVoterID()	static bool — one-time initialisation
Type conversion	generateVoterID()	to_string(), numeric_limits<streamsize>
getline() vs cin >>	main()	Mixed input — cin.ignore() idiom between them
Header files used	Throughout	<iostream>, <string>, <algorithm>, <fstream>, <ctime>, <limits>

7. Learning Outcomes

1. Translating real world legal eligibility rules into structured C++ conditional logic.
2. Using the C standard time library (time_t, struct tm, localtime()) to perform date arithmetic.
3. Implementing the canonical C++ input validation pattern: cin.fail() → cin.clear() → cin.ignore().
4. Generating pseudo-random identifiers with rand()/srand() and understanding their limitations.
5. Designing modular, single-responsibility functions in a procedural C++ codebase.
6. Handling the mixed cin >> / getline() input pattern correctly using cin.ignore().

8. Conclusion

The implementation successfully fulfils all stated requirements. Its primary technical merit lies in the calculateAge() function — which correctly handles the birthday-not-yet-passed edge case using struct tm — and the robust validation loops that prevent the program from crashing on malformed input. The use of ios::app ensures that records accumulate correctly across multiple program sessions.

Overall Assessment: The program demonstrates a strong command of foundational C++ I/O, control flow, modular design, and the standard library's time and file facilities. It is a well rounded solution to the case study requirements and provides a sound basis for extension into more advanced topics such as object oriented design, database integration, or multi-threaded processing.